

CS-405: Deep Learning

Week 5 - Advanced Computer Vision

Dr. Sajjad Hussain

SEECS, NUST

February 17th, 2026

Lecture Outline

- 1 Object Detection Overview
- 2 Sliding Window Approach
- 3 IoU and Non-max Suppression
- 4 Semantic Segmentation
- 5 Transpose Convolutions
- 6 U-Net Architecture

What is Object Detection? A Simple Analogy

Think of it like a "Spot the Object" game:

- 1 What is it? (Classification)
- 2 Where is it? (Localization)

Formal Definition

Object Detection = Localization + Classification

Given an image, identify:

- What objects are present (class labels: cat, dog, car, etc.)
- Where they are located (bounding box coordinates)

Visualizing Object Detection Output

Classification

- Class label
- Confidence score

Localization

- Bounding box
- Coordinates
(x,y,w,h)

Example Output

- "Dog: 0.95"

Key Challenge

Unlike classification (one answer per image), detection needs to find **multiple objects** of **different classes** at **different locations**!

From Classification to Detection: Adapting Networks

Step-by-step adaptation process: **Original Classification Network:**

- Input: Entire image
- Output: Single class label
- No location information

Modified Detection Network:

- Output: Multiple predictions
- Each prediction: class + box

From Classification to Detection: Adapting Networks

Step-by-step adaptation process: Original Classification Network:

- Input: Entire image
- Output: Single class label
- No location information

Modified Detection Network:

- Output: Multiple predictions
- Each prediction: class + box

The Modified Output Vector

For each potential object, we predict:

$$\text{Prediction} = [\underbrace{p_{\text{obj}}}_{\text{Object?}}, \underbrace{c_1, c_2, \dots, c_K}_{\text{Which class?}}, \underbrace{b_x, b_y, b_h, b_w}_{\text{Where?}}]$$

Example: $[0.95, 0, 1, 0, 0.3, 0.4, 0.2, 0.1] \rightarrow \text{"Cat at (0.3,0.4) with size (0.2,0.1)"}$

Modified Detection Network

Detecting a single object

“What”

Correct label:

Cat

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Softmax
Loss

Multitask
Loss

Weighted
Sum

Loss

Vector:
4096

Fully
Connected:
4096 to 4

Box
Coordinates
(x, y, w, h)

L2 Loss

Correct box:
(x', y', w', h')

Treat localization as a
regression problem!

“Where”

Figure 1: Using a pre-trained classifier for object detection

Limitations of Pre-trained Networks for Multi-Object Detection

Why single forward pass with pre-trained detector fails:

Key Problems

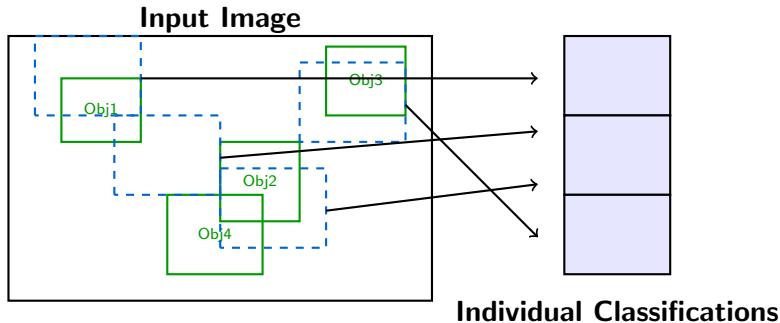
- ① **Fixed input size** assumption
- ② **Single object** bias from training
- ③ **Spatial invariance** failure
- ④ **Scale variation** issues

The Core Issue

Pre-trained detectors learn: **One image** $\rightarrow$ **One prediction**
But we need: **One image** $\rightarrow$ **Multiple predictions**

The Sliding Window Solution: Intuition

Core idea: Convert the problem from "one image $\rightarrow$ one prediction" to "many crops $\rightarrow$ many predictions"



Key Insight

Each window becomes a valid input to the pre-trained network
 $\Rightarrow$ Network sees what it was trained on: one object per input

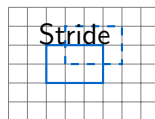
The Sliding Window Idea: A Visual Explanation

Think of it like a "Search Light"

- 1 Take a small window (like a magnifying glass)
- 2 Move it systematically across the image
- 3 At each position, ask: "Is there an object here?"
- 4 If yes, record what and where

Parameters we control:

- Window size (how big is our view?)
- Stride (how far do we move each step?)
- Scale (how zoomed in are we?)



The Scale Problem: Multi-scale Detection

Challenge: Objects come in different sizes!

Small Window



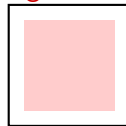
- Good for small objects
- Misses large objects

Medium Window



- Good for medium objects
- May miss extremes

Large Window



- Good for large objects
- May miss small ones

Solution: Image Pyramid

Resize image to multiple scales and run sliding window on each!

The Computational Cost Problem

Let's Count the Operations

For a 224×224 image with 3×3 sliding window:

$$\begin{aligned}\text{Windows per scale} &= \frac{\text{Image Size}}{\text{Stride}^2} \\ &= \frac{224 \times 224}{2 \times 2} \approx \mathbf{12,544} \text{ windows!}\end{aligned}$$

Multiply by number of scales (say 5):

$$12,544 \times 5 = \mathbf{62,720} \text{ classifications per image!}$$

The Computational Cost Problem

Let's Count the Operations

For a 224×224 image with 3×3 sliding window:

$$\begin{aligned}\text{Windows per scale} &= \frac{\text{Image Size}}{\text{Stride}^2} \\ &= \frac{224 \times 224}{2 \times 2} \approx \mathbf{12,544} \text{ windows!}\end{aligned}$$

Multiply by number of scales (say 5):

$$12,544 \times 5 = \mathbf{62,720} \text{ classifications per image!}$$

The Efficiency Problem

62,720 forward passes through a neural network for ONE image!

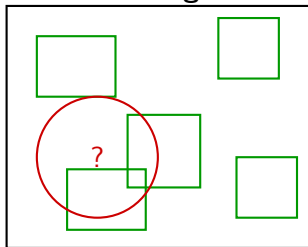
This is why we need more efficient approaches like YOLO.

Hands-on Example: Before and After Sliding Window

Experimental setup:

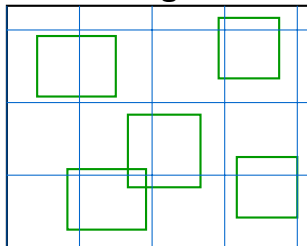
- Image: 800×600 with 5 objects
- Pre-trained detector: Trained on 224×224 images with single objects

Without Sliding Window:



Detected: 1 object (maybe)
4 objects missed

With Sliding Window:



Detected: All 5 objects
Each in its own window

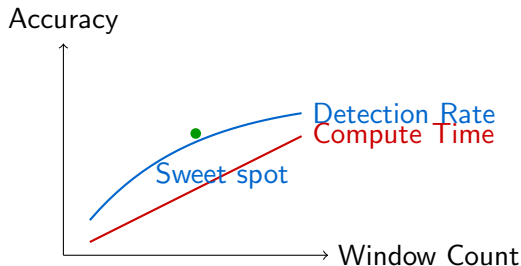
Trade-offs: Sliding Window Pros and Cons

Advantages

- + Works with pre-trained models
- + Simple to implement
- + Guaranteed coverage
- + Multi-scale capability
- + No retraining needed

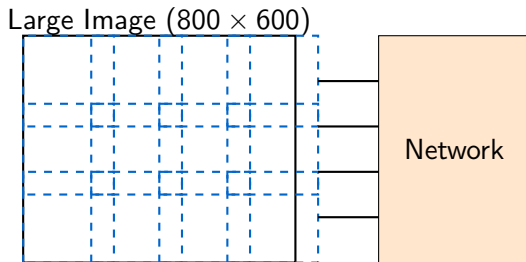
Disadvantages

- Computationally expensive
- Fixed window shapes
- Many redundant computations
- Difficult aspect ratios
- Post-processing overhead



The Challenge: Processing Larger Images

Traditional sliding window approach:



Example:

- Image: 800×600 , Window: 224×224 , Stride: 32
- Windows: $\lfloor \frac{800-224}{32} + 1 \rfloor \times \lfloor \frac{600-224}{32} + 1 \rfloor = 19 \times 13 = 247$
- With 3 scales: $247 \times 3 = 741$ forward passes!

Key Insight: FC Layers as Convolutions

Key observation: The original network's FC layers impose two critical constraints:

- **Fixed input size:** They require the input to be exactly 32×32 pixels
- **Single prediction:** They output only one classification per forward pass

By converting FC layers to CONV layers, we break both constraints:

- The network can now process **arbitrarily large images** in a single forward pass
- It produces a **spatial grid of predictions** - one for every 32×32 region
- All sliding window computations happen **simultaneously and efficiently**

Step 1: Original FC Layer Architecture

Fixed-size classification network with fully connected layers:

Layer	Output Shape	Parameters
Conv Layers	$7 \times 7 \times 512$	-
Flatten	25088	-
FC1	4096	25088×4096
FC2	4096	4096×4096
FC3	1000	4096×1000

Key Characteristics of FC Layers

- Expect fixed-size input (7×7 spatial feature map)
- Produce single vector output (lose spatial information)
- Cannot process larger images without cropping/rescaling

Step 2: Converting FC1 (25088 $\rightarrow$ 4096) to Convolution

FC1 Original Specifications:

- Input: Flattened vector of 25088 features
- Output: 4096 neurons
- Parameter count: 25088×4096

Converted Convolutional Layer Specifications:

Conv6: 4096 filters of dimension $7 \times 7 \times 512$ (kernel matches 7×7 input, 512 in $\rightarrow$ 4096 out), stride 1, padding 0

Output Shape Calculation:

$$\text{Height} = (7 - 7 + 1) = 1, \quad \text{Width} = (7 - 7 + 1) = 1$$

$$\text{Output shape} = 1 \times 1 \times 4096$$

Result: Conv6 Layer

Conv6: 4096 filters of $7 \times 7 \times 512$ dimension

Parameters: $4096 \times 7 \times 7 \times 512$

Output shape: $1 \times 1 \times 4096$

Step 3: Converting FC2 with 1×1 Convolutions

Current state after Conv6: $1 \times 1 \times 4096$ feature map

Converting FC2 ($4096 \rightarrow 4096$):

Parameter	Value
Layer name	Conv7
Kernel size	$1 \times 1 \times 4096$ (pointwise convolution)
Input channels	4096
Output channels	4096
Stride	1
Padding	0

Result: Conv7 Layer

Conv7: 4096 filters of size $1 \times 1 \times 4096$

Parameters: $4096 \times 1 \times 1 \times 4096$

Output shape: $1 \times 1 \times 4096$ (spatial size preserved)

Step 4: Converting FC3 with 1×1 Convolution

Converting FC3 ($4096 \rightarrow 1000$):

Parameter	Value
Layer name	Conv8
Kernel size	1×1 (pointwise convolution)
Input channels	4096
Output channels	1000
Stride	1
Padding	0

Result: Conv8 Layer

Conv8: 1000 filters of $1 \times 1 \times 4096$

Parameters: $4096 \times 1 \times 1 \times 1000$

Output shape: $1 \times 1 \times 1000$

Complete Converted Architecture & Key Insight

Summary of Converted Layers:

Layer	Kernel	Output Shape	Parameters
Conv6	$7 \times 7 \times 512$	$1 \times 1 \times 4096$	$512 \times 7 \times 7 \times 4096$
Conv7	$1 \times 1 \times 4096$	$1 \times 1 \times 4096$	$4096 \times 1 \times 1 \times 4096$
Conv8	$1 \times 1 \times 4096$	$1 \times 1 \times 1000$	$4096 \times 1 \times 1 \times 1000$

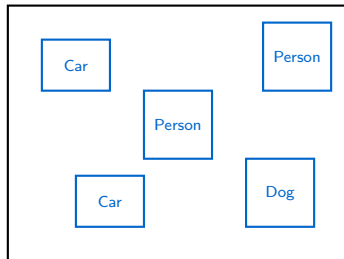
Critical Insight: Dense Detection in One Forward Pass

- **For 7×7 input:** Output is $1 \times 1 \times 1000$ (single classification)
- **For larger input (e.g., 14×14):**
 - Conv6 output: $(14 - 6) \times (14 - 6) \times 4096 = 8 \times 8 \times 4096$
 - Conv7 output: $8 \times 8 \times 4096$ (spatial size preserved)
 - Conv8 output: $8 \times 8 \times 1000$ **spatial heatmap!**
- **Interpretation:** Each of the $8 \times 8 = 64$ spatial locations in the output corresponds to one 7×7 region in the input image

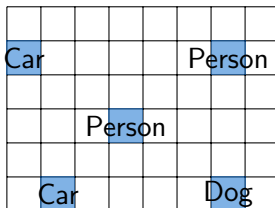
Single forward pass = All sliding window detections

Visual Example: Detection on Larger Image

Original image with multiple objects (800×600):



Output heatmap after FC→Conv conversion (8×8 grid):



Each cell = detection at corresponding location

Advantages Summary: Why This Matters

Computational Efficiency

- Single forward pass instead of hundreds
- Shared computation across overlapping windows
- GPU-friendly parallel processing
- Real-time capable (>30 FPS)

Architectural Benefits

- End-to-end differentiable
- No special post-processing
- Works with any pre-trained model
- Natural multi-scale integration

This is the foundation for modern detectors like YOLO!

Key takeaway: Converting FC to Conv transforms a **single-location classifier** into a **dense spatial detector** with a single forward pass.

From FC→Conv to YOLO: The Natural Evolution

Recall: Converting FC to Conv gave us:

- Ability to process **arbitrarily large images** in one pass
- **Spatial heatmap** output: $H \times W \times 1000$ (class scores per location)
- Each spatial location corresponds to one sliding window position

But what's missing?

- We only get **classification** per window, not bounding boxes
- Windows are **fixed size** (224×224 in our example)
- No way to handle multiple objects in the same window

YOLO's Key Insight

Why not predict **bounding boxes** along with class probabilities
for **each grid cell** in the output heatmap?

FC→Conv → **Spatial grid of predictions** → YOLO: Each grid cell
predicts **boxes + classes**

YOLO: You Only Look Once - Core Concept

Single neural network predicts everything in one forward pass:

Input: Entire image

- Divide into $S \times S$ grid (typically $S = 7$)
- Each grid cell is responsible for objects whose center falls in it

Output: $S \times S \times (B \times 5 + C)$ tensor

- $S \times S$: grid cells
- B : bounding boxes per cell (typically $B = 2$)
- 5: box coordinates (x, y, w, h) + confidence
- C : class probabilities (e.g., 20 for PASCAL VOC)

Output tensor visualized:

7×7 grid $\times$ [Box1: ($x, y, w, h, conf$), Box2:
($x, y, w, h, conf$), $p_1, p_2, \dots, p_C$]

$$\text{Total output size} = 7 \times 7 \times (2 \times 5 + 20) = 7 \times 7 \times 30$$

Decoding YOLO's Predictions

For each grid cell (i, j) , YOLO predicts:

① **Bounding Boxes ($B = 2$):**

- x, y : Center coordinates *relative to the grid cell* (between 0 and 1)
- w, h : Width and height *relative to the entire image* (between 0 and 1)
- **confidence** = $P(\text{Object}) \times \text{IoU}_{\text{pred}}^{\text{truth}}$

② **Class Probabilities ($C = 20$):**

- $p_1, p_2, \dots, p_C$: Probability of each class *given there's an object*
- Only one set of class probabilities per cell (shared by both boxes)

Example: One grid cell's prediction vector

Box 1					Box 2					Classes	
x	y	w	h	c	x	y	w	h	c	p_1	p_2
0.3	0.7	0.2	0.3	0.9	0.5	0.2	0.1	0.2	0.3	0.1	0.8

Final Detection Score

Score = confidence $\times$ class probability

Only keep predictions with score $>$ threshold

Preparing Training Data for YOLO

For each training image, we need to create the target tensor:

Target shape: $S \times S \times (B \times 5 + C)$

Example: $7 \times 7 \times 30$

Step-by-step labeling process:

- 1 Divide image into $S \times S$ grid (e.g., 7×7)
- 2 For each object in the image:
 - Find which grid cell contains the object's **center**
 - That grid cell becomes **responsible** for predicting this object
- 3 For each grid cell:
 - If object present: set confidence = 1, assign class
 - If no object: set confidence = 0 for all boxes

Key Rule

Each object is assigned to **exactly one grid cell** - the cell containing its center.

Multiple objects in the same cell? YOLO can handle up to B boxes per cell.

How YOLO Learns: Loss Function Components

YOLO loss combines three objectives:

- 1 **Coordinate loss** - for box coordinates (x, y, w, h)
- 2 **Confidence loss** - for objectness score
- 3 **Classification loss** - for class probabilities

Loss Component	What it does
$\lambda_{\text{coord}} \sum (x - \hat{x})^2 + (y - \hat{y})^2$	Penalizes center coordinate errors
$\lambda_{\text{coord}} \sum (\sqrt{w} - \sqrt{\hat{w}})^2 + (\sqrt{h} - \sqrt{\hat{h}})^2$	Penalizes size errors (sqrt for small
$\sum (\text{confidence} - \text{confidence})^2$	Penalizes wrong objectness predictions
$\sum (\text{class} - \hat{\text{class}})^2$	Penalizes wrong class predictions

Key weighting:

- $\lambda_{\text{coord}} = 5$ (emphasize getting boxes right)

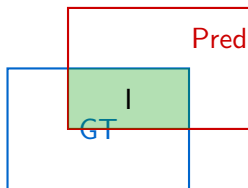
One loss function, end-to-end differentiable!

Intersection over Union (IoU): Measuring Box Overlap

Simple Question

How do we know if a predicted box is "correct"?

Answer: Measure how much it overlaps with the true box!



$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

$$\begin{aligned}\text{IoU} &= \frac{\text{Green}}{\text{Blue} + \text{Red} - \text{Green}} \\ &= \frac{2}{6 + 6 - 2} = \frac{2}{10} = 0.2\end{aligned}$$

IoU Threshold Interpretation

$$\text{IoU} > 0.5$$

Good detection

$$0.3 < \text{IoU} < 0.5$$

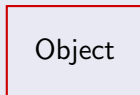
Poor localization

$$\text{IoU} < 0.3$$

False positive

Non-max Suppression (NMS): Cleaning Up Duplicates

The Problem: Multiple Detections for Same Object



NMS Algorithm Steps:

- 1 Sort all detections by confidence score
- 2 Select the box with highest confidence
- 3 Remove any box with $\text{IoU} > \text{threshold}$ with selected box
- 4 Repeat for remaining boxes

Box	Confidence
Box A	0.95
Box B	0.87
Box C	0.45
Box D	0.23

Analogy

Like picking the best photo of a group - you keep the clearest one and discard blurry duplicates!

Semantic Segmentation: Pixel-Level Understanding

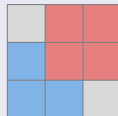
From "What's there?" to "What's at EVERY pixel?"

Object Detection:



- Boxes around objects
- "Cat here, Dog there"

Semantic Segmentation:



- Every pixel labeled
- "Cat pixels, Dog pixels, Background pixels"

Key Difference

Detection: **Sparse** (only some pixels in boxes matter)

Segmentation: **Dense** (EVERY pixel gets a label!)

Segmentation Architectures: The Encoder-Decoder Idea

The "Zoom Out, Then Zoom In" Strategy

Encoder (Zoom Out):



- Gradually reduce spatial size
- Increase feature depth
- Capture "what" (context)

Decoder (Zoom In):



- Gradually increase spatial size
- Reduce feature depth
- Recover "where" (location)

Analogy

Like reading a map: First get the big picture (encoder), then zoom in to see details (decoder).

Transpose Convolution: Learnable Upsampling

The Problem: How do we increase spatial size?

Options:

- Bilinear interpolation / Nearest neighbor (fixed, not learnable)
- **Transpose Convolution** (learnable!)

Regular Convolution:

- Input: $4 \times 4 \rightarrow$ Output: 2×2
- **Downsampling**

Transpose Convolution:

- Input: $2 \times 2 \rightarrow$ Output: 4×4
- **Upsampling**

Intuition

Think of it as the "reverse" of a convolution:

- One input value contributes to multiple output values
- The network learns the best way to upsample
- Like learning to "paint" details from compressed features

Transpose Convolution: Intuition

Core Idea

Each input pixel “paints” a scaled copy of the kernel onto a larger canvas. Overlapping painted regions are added together.

Input (2x2)

a	b
c	d

Kernel (3x3)

	W	

Step-by-Step Idea

- 1 Take one pixel (e.g., a)
- 2 Multiply entire kernel by a
- 3 Place (stamp) it on output grid
- 4 Repeat for b, c, d
- 5 Add overlapping values

Result: Larger output feature map

Transpose Convolution: Visual Mechanism

Example: Only pixel a active (others = 0)

aw_{11}			
aw_{22}			
aw_{33}			

Now when b, c, d also stamp their kernels, overlapping regions are **summed together**.

Key Insight

Normal Convolution: Slide kernel over input (shrinks size)

Transpose Convolution: Spread input over output (expands size)

Output Size of Transpose Convolution

Output Size Formula

$$H_{out} = (H_{in} - 1) \times \text{stride} + K$$

Why?

- Each input pixel produces a kernel-sized patch (K)
- Neighboring patches are spaced by stride
- There are $(H_{in} - 1)$ gaps between pixels
- So total size grows accordingly

Intuition: Kernel size determines patch size, stride determines spacing.

U-Net: The Breakthrough in Medical Segmentation

Why U-Net was Revolutionary

Problem with simple encoder-decoder:

- Encoder loses spatial details (where things are)
- Decoder struggles to recover precise boundaries

U-Net solution:

- Add **skip connections** between encoder and decoder
- Like taking shortcuts to preserve spatial information

UNet

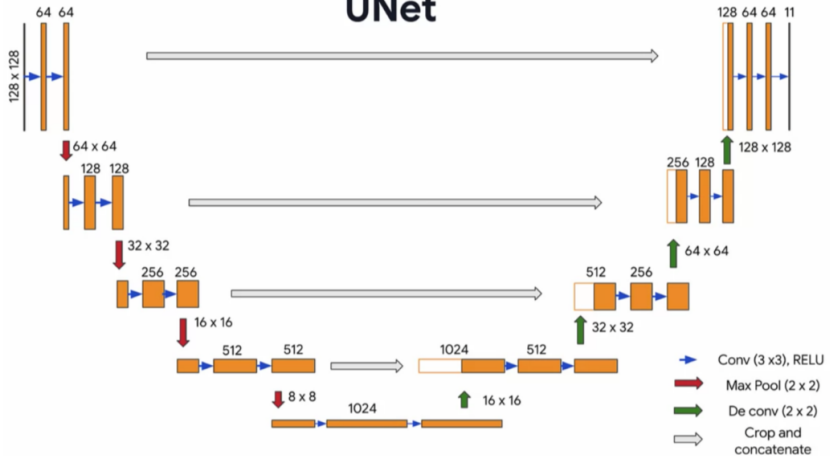


Figure 2: UNet Architecture for Image Segmentation

U-Net Architecture: Detailed Walkthrough

Contracting Path (Encoder)

- 1 3x3 conv + ReLU (x2)
 - 2 2x2 max pool
 - 3 Double feature channels
- Learn features
 - Reduce size
 - Increase depth

Bottleneck

- 1 3x3 conv + ReLU
 - 2 Most compressed
 - 3 Bridge to decoder
- Deepest features
 - Global context

Expanding Path (Decoder)

- 1 2x2 up-conv
 - 2 Concatenate with encoder features
 - 3 3x3 conv + ReLU (x2)
- Increase size
 - Combine features
 - Precise localization

Why Skip Connections Matter

Encoder knows **WHAT** (high-level features)

Decoder needs **WHERE** (spatial details)

Skip connections bring WHERE from encoder to decoder!

U-Net Applications Beyond Medical Imaging

Medical

- Tumor seg.
- Organ seg.
- Cell seg.

Autonomous

- Road seg.
- Lane detection
- Obstacle seg.

Satellite

- Land cover
- Building seg.
- Road extraction

Creative

- Background removal
- Image editing
- Style transfer

Why So Popular?

U-Net works well with **limited data** and produces **sharp boundaries!**

Summary and Key Takeaways

Object Detection

- Localization + Classification
- YOLO: Single-shot grid-based
- Post-processing: IoU + NMS

Semantic Segmentation

- Pixel-wise classification
- Encoder-decoder architecture
- U-Net: Skip connections

Next Steps

- Practice with pre-trained models
- Experiment with different architectures
- Explore instance segmentation (Mask R-CNN)